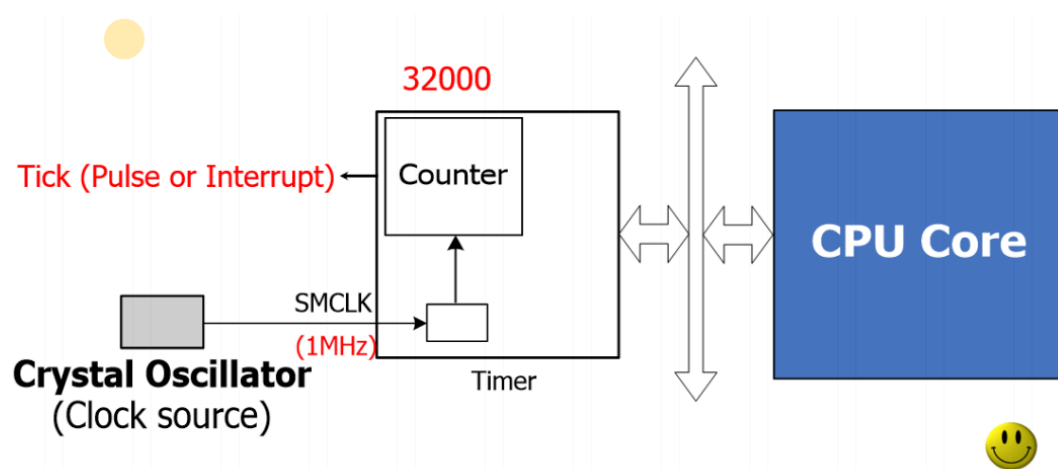


嵌入式期末试卷

一、计算题（1题，8分）

1.给出了下面的图，给出了1MHz的频率，问如果需要每32ms一个tick的话，Counter的值应该设置为多少



在文件 "image_56e259.png" 中，这道题考察的是底层硬件定时器（Timer）计数值（Counter）的计算。图中的红色标注 **32000** 就是正确的 Counter 设置值。

下面是推导这个结果的详细计算过程：

1. 提取已知条件

- **时钟源频率 (SMCLK):** $f = 1 \text{ MHz} = 1,000,000 \text{ Hz}$ 。这意味着时钟晶振每秒钟会产生 1,000,000 个脉冲。
- **目标 Tick 周期:** $T_{\text{tick}} = 32 \text{ ms} = 0.032 \text{ s}$ 。这是我们需要定时器溢出并产生一次中断（Interrupt）的时间间隔。

2. 计算过程

我们需要计算出在 32 毫秒的时间窗口内，包含了多少个时钟脉冲周期。公式如下：

$$\text{Counter} = f \times T_{\text{tick}}$$

将已知数据代入公式：

$$\text{Counter} = 1,000,000 \text{ Hz} \times 0.032 \text{ s} = 32,000$$

因此，将 Counter 寄存器的值设置为 32000，定时器就会每隔 32ms 向 CPU 发送一个 Tick。

底层系统中的实际应用

掌握这个计算逻辑对于嵌入式开发和操作系统底层原理至关重要，这也往往是期中考试里非常经典的考点。

- **RTOS 系统心跳**：在配置实时操作系统（如 uC/OS-II 或 aCoral）时，第一步往往就是配置此类硬件定时器来产生 SysTick。内核完全依赖这个精准的周期性中断来进行任务调度（Task Scheduling）、延时管理和时间片轮转。
- **算法采样控制**：在开发需要精确控制的硬件时，这类定时器用来保证每次误差采样的频率是绝对固定的。例如在使用 PID 算法控制硬件系统时，只有保证 Δt 恒定不变，公式中的积分和微分项计算才会有意义。

二、简答题(5题，50分)

1. 嵌入式实时系统采用了哪些机制来保证实时性和确定性？（10分）

这是一道非常经典的嵌入式操作系统期中考试简答题。要拿满这 10 分，你需要从**调度、中断、同步、内存和时间**这五个内核基础核心维度来进行条理清晰的阐述。

嵌入式实时系统（如 aCoral、uC/OS-II、VxWorks）为了保证**实时性**（必须在严格的截止时间内完成操作）和**确定性**（系统的每一次动作耗时都是可预测的，最坏情况有上限），主要采用了以下几种核心机制：

1. 基于优先级的抢占式调度机制 (Priority-based Preemptive Scheduling)

- **机制实现**：内核剥夺了任务对自己执行时间的绝对控制权。系统总是让就绪队列中优先级最高的任务获得 CPU。一旦有更高优先级的任务就绪，正在运行的低优先级任务会被立刻打断（抢占）。
- **确定性保证**：为了保证调度器本身的耗时是确定的，优秀的 RTOS 通常会采用查表法（如 uC/OS-II 的就绪表位图）或利用处理器的前导零计算硬件指令，使得寻找最高优先级任务的时间复杂度保持为 $O(1)$ 。这意味着无论系统有 10 个任务还是 100 个任务，调度算法的耗时都是恒定且已知的。

2. 短且可预测的中断响应机制 (Deterministic Interrupt Handling)

- **机制实现**：实时系统的内核设计会尽可能缩短“关中断”的时间（即临界区尽量短小精悍）。同时，支持中断嵌套，允许高优先级的硬件中断抢占低优先级的硬件中断。
- **确定性保证**：这使得系统的“中断延迟（Interrupt Latency）”极小且具有严格的计算上限。此外，通常提倡将中断处理分为两部分（例如 ISR 加上延迟的服务任

务)，在 ISR 中只做最少量的寄存器读写和标志位清除，耗时的数据处理交由任务去做，防止长时间屏蔽其他中断，保证了系统的微秒级响应。

3. 解决“优先级反转”的任务同步机制 (Priority Inversion Avoidance)

- **机制实现：** 在多任务共享互斥资源（如使用 Mutex）时，极易出现低优先级任务持有锁，导致高优先级任务被迫等待，甚至被中等优先级任务长期抢占的“优先级反转”致命问题。实时系统通常引入**优先级继承协议 (Priority Inheritance Protocol)** 或**优先级天花板协议 (Priority Ceiling Protocol)**。
- **确定性保证：** 当高优先级任务被阻塞时，系统会临时提升占有资源的低优先级任务的优先级，使其尽快执行完毕并释放资源。这保证了高优先级任务的阻塞等待时间是确定的 (Boundable)，避免了不可控的延迟或死锁。

4. 无碎片且时间恒定的内存管理机制 (Deterministic Memory Management)

- **机制实现：** 通用操作系统中的 `malloc` 和 `free` 会产生内存碎片，且在碎片堆中寻找可用内存块的耗时是不确定的。RTOS 为了避免这个问题，通常提倡静态分配内存，或者采用**定长内存池 (Fixed-size Memory Blocks)** 进行动态分配。此外，RTOS 极少使用虚拟内存 (MMU 页面映射)。
- **确定性保证：** 因为没有虚拟内存，就彻底杜绝了 Page Fault（缺页中断）带来的巨大且不可控的耗时。而固定大小的内存池使得分配和释放一块内存的时间也是 $O(1)$ 的，彻底消除了内存管理带来的时间不可预测性。

5. 高精度的时间节拍与定时机制 (Precise Time Management)

- **机制实现：** 就像底层硬件的 Crystal Oscillator 与 Timer 模块所设定的那样，RTOS 依赖硬件定时器产生精准的系统心跳 (SysTick)，维持整个内核的时间轴运转。
- **确定性保证：** 提供高精度的相对延时 (Relative Delay) 和绝对延时 (Absolute Delay) API。这确保了周期性任务（例如智能车上严格要求 Δt 恒定的 PID 闭环控制算法）能够在极其精确的时间点被唤醒并执行采样计算。

考试答题建议：

在答题纸上列出这五点作为“踩分点”后，可以稍微结合你们课上重点学习的具体系统（比如提一句“类似 uC/OS-II 中的 $O(1)$ 调度算法”或“aCoral 中的互斥量设计”），这样能充分展示对底层原理的具体掌握，基本就能稳拿这 10 分了。

2.按照实时性划分，嵌入式实时系统可以划分为什么，分别叙述其特点和应用领域 (8分)

这同样是一道非常经典的 RTOS 理论必考题。按照对“截止时间”(Deadline) 要求的严格程度，嵌入式实时系统通常被划分为**硬实时系统**和**软实时系统**两大类（在某些学术体系中还会加入**坚实实时系统**作为补充）。

为了稳拿这 8 分，建议你在答题纸上按照以下结构清晰地分点作答：

1. 硬实时系统 (Hard Real-Time Systems)

- **特点 (Characteristics):**

- **绝对的确定性：** 系统必须在严格规定的、绝对的截止时间内完成任务。
- **灾难性后果：** 如果任务超时（哪怕只是晚了一毫秒），系统不仅是性能下降，而是会导致极其严重的、不可挽回的灾难性后果（如人员伤亡、重大财产损失或系统彻底崩溃）。
- **关注点：** 重点在于系统的最坏情况响应时间（Worst-Case Execution Time, WCET）是可预测且有绝对上限的。

- **应用领域 (Applications):**

- 高度安全关键的领域。例如：航空航天飞行控制系统、核电站反应堆控制、医疗生命维持设备（如心脏起搏器）、汽车的安全气囊与防抱死制动系统 (ABS)。
- 在智能小车的底层开发中，负责读取编码器数据并计算输出给电机的 **PID 闭环控制循环**也是典型的硬实时任务——如果定时器中断漂移、采样周期无法保证，小车极易直接冲出赛道或翻车。在 **F1 赛车的引擎控制单元 (ECU)** 中，点火和喷油的时机控制也是极其严苛的硬实时需求。

2. 软实时系统 (Soft Real-Time Systems)

- **特点 (Characteristics):**

- **统计学意义上的确定性：** 系统通常也要求在截止时间内完成任务，但这个期限是有弹性的。
- **非致命后果：** 如果偶尔错过了截止时间，系统不会崩溃，也不会造成灾难性后果。超时仅仅会导致系统性能下降、服务质量 (QoS) 降低或者用户体验变差。
- **关注点：** 重点在于系统的平均响应时间（Average Response Time）和整体吞吐量，允许偶发的延迟。

- **应用领域 (Applications):**

- 多媒体与信息交互领域。例如：视频会议流媒体播放（偶尔丢帧只会导致画面卡顿）、在线数据库查询系统、自动取款机 (ATM) 网络、以及网页服务器。
- 在**智能小车系统的开发中**，负责将小车状态通过无线串口发送给电脑，并在**上位机 UI 界面（如基于 Tauri 写的仪表盘）上刷新显示**的任务，就属于软实时——哪怕界面偶尔卡顿了一下没有及时更新，也不会影响小车自身的物理运动安全。

 加分项提示（坚实时系统 - Firm Real-Time）:

如果你想让答案看起来更专业（视你们课本和老师上课的内容而定），可以补充一两句话提及**坚实时系统**：

*"介于两者之间的还有**坚实时系统**。它的特点是：超时不会导致灾难性物理破坏，但超时后计算出的结果将失去任何价值（价值瞬间降为 0），系统会直接丢弃该结果。典型应用如：高频量化交易系统（晚了一秒钟的行情数据就毫无意义了，但不会导致服务器爆炸）或气象预报系统。"*

按这个框架把“特点”和“应用”对比着写，条理分明，阅卷老师一眼就能看到所有的得分点。

3.在嵌入式实时系统中，如何体现根据特定需求定制H/W和S/W？（大概这意思）
(8分)

这也是一道考察对嵌入式系统本质理解的核心题目。嵌入式系统最根本的特征就是“以应用为中心，软硬件可裁剪”。为了拿满这 8 分，你需要分别从硬件（H/W）和软件（S/W）两个维度，阐述系统是如何做到“量体裁衣”的。

建议在答题时采用总分结构，以下是高分答题框架：

核心理念 (总起)

在嵌入式实时系统中，由于严格的成本、体积、功耗和实时性限制，系统设计遵循“够用就行 (Just Enough)”原则。软硬件的定制是紧密耦合的（软硬件协同设计），完全基于最终应用场景的特定需求进行裁剪。

1. 硬件 (H/W) 层的定制体现 (约 4 分)

硬件定制的核心在于**去冗余**，只保留完成特定任务所需的最小物理资源集合。

- **处理器的精准选型 (Processor Selection):**

不追求绝对的高性能，而是追求“最合适”。例如，如果需求只是简单的逻辑控制和定时，可能会选择 8 位单片机；如果需求是运行复杂的 PID 闭环控制算法或处理

大量中断，则会定制带有 FPU（浮点运算单元）或特定硬件乘法器的 32 位微处理器（如基于 ARM 架构的芯片）。

- **外设接口的按需集成 (Peripheral Tailoring):**

通用计算机包含所有可能用到的接口（USB, HDMI, 蓝牙等）。而在嵌入式硬件定制中，完全取决于挂载的传感器和执行器。例如，在设计底层运动控制板时，如果需要驱动多路电机，就会在硬件上定制丰富的 PWM 输出通道和高精度的 Timer；如果不需要屏幕显示，就会彻底舍弃显卡和显示接口芯片，从而节省成本和功耗。

- **存储容量的精确计算 (Memory Sizing):**

嵌入式系统的 ROM (Flash) 和 RAM 通常非常有限（有时只有几 KB 到几十 KB）。硬件工程师会根据最终编译出的二进制代码大小以及系统运行所需的最大堆栈深度，来严格定制存储芯片的容量，绝不堆砌无用的内存。

- **物理与环境特性的定制 (Physical & Environmental Constraints):**

根据工作环境定制硬件的物理形态。例如，防震抗摔的工业级封装、耐高温的汽车级芯片、或者针对电池供电设备定制的超低功耗电源管理模块。

2. 软件 (S/W) 层的定制体现 (约 4 分)

软件定制的核心在于**可裁剪性 (Scalability)**，确保操作系统和应用程序的代码体积最小、运行效率最高。

- **RTOS 内核的深度裁剪 (Kernel Scalability):**

这是最典型的软件定制体现。在使用源码级的实时操作系统（如 uC/OS-II、aCoral 或 VxWorks）时，开发者可以通过配置头文件（宏定义），将系统裁剪到极简状态。如果特定项目不需要“事件标志组”或“动态内存分配”，可以直接将对应的宏设为 0。编译时这些内核代码将被全部剔除，从而将内核体积从几十 KB 压缩到几 KB。

- **板级支持包的定制 (BSP - Board Support Package):**

因为硬件是定制的，底层的驱动程序也必须完全定制。软件开发人员需要针对这块独一无二的定制电路板，编写特定的硬件初始化代码、中断向量表和设备驱动（如定制的串口驱动、ADC 驱动），使操作系统能够顺畅运行在特定的硬件上。

- **任务调度的针对性设计 (Application-Specific Scheduling):**

软件工程师会根据特定需求的实时性约束，手工定制任务的数量、优先级和时间片。绝对保证关键任务（如底层传感器的高频采样、安全告警）获得最高优先级，并根据硬件的运算能力优化算法（例如，如果硬件不支持浮点运算，就在软件层定制一套定点数运算算法来替代）。

- 启动引导程序的极简设计 (Bootloader Customization):

PC机的 BIOS 会进行漫长而全面的硬件自检，而嵌入式系统的 Bootloader 通常被定制得极其精简，只初始化最核心的硬件（如时钟、内存），然后立刻将控制权交给主程序，以满足特定的“快速启动”需求。

💡 答题小贴士:

在考场上，可以这样总结一句作为收尾：“总而言之，通用计算机是先有标准化的软硬件，再让用户去寻找应用；而嵌入式系统是先有明确的特定应用需求，再去反向定制和裁剪最合适的软硬件架构。”这句话往往能让老师眼前一亮。

4.给了如下的代码，让你解释中断响应流程 (10分)

```
IRQ
    stmdb    sp!, {r0-r2}                ;sp_irq
    mov      r0, sp
    add      sp, sp, #12
    sub      r1, lr, #4                  ;lr_irq
    mrs      r2, spsr
    msr      cpsr_cxsf, #SVCMODE|NOINT
    stmdb    sp!, {r1}                  ;sp_svc
    stmdb    sp!, {r3-r12, lr}
    ldmia    r0!, {r3-r5}
    stmdb    sp!, {r2-r5}
    ldr      r0, =OSIntNesting
    ldr      r1, [r0]
    add      r1, r1, #1
    strb     r1, [r0]
    teq      r1, #1
    bne      %F1
    ldr      r0, =OSTCBCur
    ldr      r0, [r0]
    str      sp, [r0]
1   msr      psr_c, #IRQMODE|NOINT
    ldr      r0, =INTOFFSET              ;10:    0x28
    ldr      r0, [r0]
    ldr      r1, =HandleEINT0           ;0x33FFFF20
    mov      lr, pc
```

```

ldr    pc, [r1, r0, lsl #2]
msr    cpsr_c, #SVCMODE | NOINT
bl     OSIntExit
ldr    r0, [sp], #4
msr    spsr_cxsf, r0
ldmia  sp!, {r0-r12, lr, pc}^

```

这是一道非常经典的基于 ARM9（S3C2440）和 uC/OS-II 操作系统的中断处理流程分析题。这段汇编代码展示了从硬件触发 IRQ 中断到最终返回任务的完整底层逻辑。

为了拿到这 10 分，你需要将整个流程清晰地划分为**四个主要阶段**来进行解释：**保护现场、中断嵌套处理、执行中断服务函数、退出与恢复现场**。

以下是详细的代码解释流：

1. 保护现场 (Context Saving)

在这个阶段，系统需要将当前被打断的任务的状态（寄存器）完整地保存到该任务自己的堆栈（SVC 模式堆栈）中。

- `stmdb sp!, {r0-r2}` 到 `add sp, sp, #12`：当 IRQ 发生时，处理器进入 IRQ 模式。这几句代码先借助 IRQ 模式的堆栈临时保存 `r0-r2`，然后用 `r0` 记录下临时保存的地址，最后恢复 IRQ 的堆栈指针（相当于没占用 IRQ 堆栈，避免 IRQ 堆栈溢出）。
- `sub r1, lr, #4`：**关键点**。计算真正的返回地址。ARM 流水线机制导致 IRQ 的 `lr` 寄存器保存的是被打断指令的“下一条再下一条”地址，所以必须减 4 才能回到被中断的精确位置。
- `mrs r2, spsr`：把 `spsr`（保存的程序状态寄存器，即被打断任务的 `cpsr`）读到 `r2` 中。
- `msr cpsr_cxsf, #SVCMODE | NOINT`：**切换模式**。关闭中断，并强行将处理器切换到 **SVC（管理）模式**。因为 uC/OS-II 的任务堆栈都在 SVC 模式下，我们必须把上下文保存在任务自己的堆栈里。
- `stmdb sp!, {r1}` 到 `stmdb sp!, {r2-r5}`：在 SVC 模式下，将返回地址（`r1`）、通用寄存器（`r3-r12, lr`）压栈。接着用 `ldmia` 把刚才在 IRQ 堆栈里暂存的 `r0-r2` 取出来，连同 `spsr`（当前的 `r2`）一起，使用 `stmdb` 压入 SVC 堆栈。

- 至此，被打断任务的所有 CPU 上下文（R0-R12, LR, PC, CPSR）已全部妥善保存在了当前任务的堆栈中。

2. 中断嵌套记录与保存栈顶 (Interrupt Nesting & TCB Update)

操作系统需要知道当前是第几层中断，以此来决定是否需要更新任务控制块（TCB）。

- `ldr r0, =OSIntNesting` 到 `strb r1, [r0]`：将 uC/OS-II 的全局变量 `OSIntNesting`（中断嵌套层数）加 1。
- `teq r1, #1` / `bne %F1`：判断是不是第一层中断。如果不是第一层中断（发生中断嵌套，跳转到标签 1），说明 CPU 原本就是在处理中断，之前的任务堆栈指针已经保存过了，不需要再保存。
- `ldr r0, =OSTCBCur` 到 `str sp, [r0]`：如果是第一层中断，说明刚刚打断的是一个正常的任务。此时必须提取当前任务控制块的指针 `OSTCBCur`，并将当前的堆栈指针 `sp`（刚刚存满上下文的地址）保存到当前任务的 TCB 中。这是后续系统做任务调度的关键依据。

3. 分发并执行中断服务函数 (ISR Dispatching)

这里体现了硬件与软件结合的查表跳转机制。

- `1 msr psr_c, #IRQMODE|NOINT`：切回 IRQ 模式（或者在某些实现中保持 SVC）。
- `ldr r0, =INTOFFSET` / `ldr r0, [r0]`：读取 S3C2440 特有的硬件中断偏移寄存器 `INTOFFSET`。这个寄存器会指出当前到底是哪个硬件外设（比如定时器、串口）触发了中断。
- `ldr r1, =HandleEINT0`：获取中断向量表（函数指针数组）的首地址。
- `mov lr, pc`：将当前的 `pc` 赋值给 `lr`，为接下来的 C 函数调用准备好返回地址。
- `ldr pc, [r1, r0, lsl#2]`：（结合图2的第一句）利用基地址 `r1`，加上硬件提供的偏移量 `r0`（乘以 4，因为函数指针占 4 字节，`lsl#2` 就是左移 2 位），计算出对应的 C 语言中断服务函数（ISR）地址，并直接赋值给 `pc`，实现跳转执行。

4. 退出中断与恢复现场 (Interrupt Exit & Context Restore)

中断服务函数执行完毕返回后，进行收尾和系统调度工作。

- `msr cpsr_c, #SVCMode|NOINT`：确保 CPU 处于 SVC 模式，且中断关闭状态。
- `bl OSIntExit`：非常关键的一步。调用 uC/OS-II 的中断退出函数。这个函数内部会将 `OSIntNesting` 减 1。如果减到 0，说明所有中断都处理完了，它会检查是否有比被中断任务更高优先级的任务就绪。如果有，`OSIntExit` 会直接触发一次任务调度（更改 `OSTCBCur` 并调整 `SP`）。

- `ldr r0, [sp], #4 / msr spsr_cxsf, r0`：从堆栈中弹出之前保存的 `spsr`（状态寄存器），并写入当前模式的 `spsr` 中。
- `ldmia sp!, {r0-r12, lr, pc}^`：这是整个流程的最后一句。从堆栈中弹出所有保存的通用寄存器和返回地址（PC）。注意最后的符号 `^`，它的作用是告诉 CPU，在恢复 `pc` 的同时，自动将刚才恢复的 `spsr` 强制拷贝回 `cpsr`。
 - 这一句指令实现了原子操作：既恢复了寄存器，又恢复了被打断任务的运行模式（如切回 User 或 SYS 模式）并顺带打开了中断，程序无缝继续运行。

考试答题技巧：答题时不需要把每一行代码都翻译一遍，阅卷老师想看的是你对**整体逻辑框架**的理解。把上面这四个大标题和关键寄存器操作（特别是 `sub r1, lr, #4`、`OSTCBCur` 的保存、`OSIntExit` 的作用，以及最后的 `^` 符号恢复状态）写清楚，就能拿到满分。

5.解释实模式到保护模式切换的流程(画流程图并解释)，并对下面代码的作用做详细解释(14分)

```
xor eax, eax
mov ax, cs
shl eax, 4
add eax, LABEL_SEG_CODE32
mov word [LABEL_DESC_CODE32 + 2], ax
shr eax, 16
mov byte [LABEL_DESC_CODE32 + 4], al
mov byte [LABEL_DESC_CODE32 + 7], ah
```

这是一道非常经典的 x86 操作系统底层开发考题，通常出自类似《操作系统真象还原》或《Orange's: 一个操作系统的实现》这样的教材。这 14 分主要考察你对 **x86 运行模式切换原理**以及 **GDT（全局描述符表）内存结构**的深入理解。

下面为你拆解这道题的满分答题框架：

第一部分：实模式到保护模式的切换流程（约 7 分）

在试卷上，你可以用文字框和箭头来绘制流程图。

1. 切换流程图

Plaintext



2. 流程详细解释（踩分点）

- 1. 准备 GDT：** 在内存中构建全局描述符表（Global Descriptor Table），定义好保护模式下需要的代码段、数据段、栈段等段描述符。
- 2. 加载 GDTR：** 也就是把 GDT 在内存中的起始物理地址和界限大小，通过 `lgdt` 指令加载到 GDTR（全局描述符表寄存器）中，让 CPU 知道 GDT 在哪。
- 3. 打开 A20 地址线：** 实模式下为了兼容 8086，默认开启了地址回绕（第 21 根地址线 A20 恒为 0）。要访问 1MB 以上的物理内存，必须通过操作端口（如 0x92 或 0x60 键盘控制器）打开 A20 门。
- 4. 关中断：** 执行 `cli` 指令禁用可屏蔽硬件中断。因为此时实模式的 IVT（中断向量表）已经不能用了，而保护模式的 IDT（中断描述符表）还没建好，一旦发生中断系统必死机。
- 5. 置 PE 位：** 将 CPU 的 `CR0` 控制寄存器的第 0 位（PE 位，Protection Enable）设为 1。这是真正在硬件层面上开启保护模式的开关。
- 6. 远跳转清空流水线：** 刚把 PE 位置 1 时，CPU 的指令预取队列里还混杂着按实模式 16 位译码的指令。必须立刻执行一条远跳转指令（如 `jmp dword Selector:Offset`），强制 CPU 清空流水线，并把新的 32 位代码段选择子加载到 CS 寄存器中，彻底完成模式切换。

第二部分：汇编代码详细解释（约 7 分）

文件 "image_5603e1.png" 中的这段汇编代码，其核心作用是：**在实模式下，动态计算出 32 位保护模式代码段（`LABEL_SEG_CODE32`）的绝对物理地址，并将其拆分填入对应的 GDT 段描述符（`LABEL_DESC_CODE32`）的 Base（基地址）字段中。**

为什么要这样做？因为编译器在编译时只知道偏移量，不知道这段代码最终被加载到物理内存的哪个确切段地址，所以必须在运行时通过当前的 `cs` 寄存器来动态计算。

逐行代码详细解释：

- `xor eax, eax`

清空 `eax` 寄存器，将其设为 0。为了防止高 16 位有垃圾数据干扰后续计算。

- `mov ax, cs`

将当前实模式下的代码段寄存器 `cs` 的值传给 `ax`。

- `shl eax, 4`

将 `eax` 左移 4 位。这是实模式下计算物理地址的经典公式： $\text{物理地址} = \text{段地址} \times 16 + \text{偏移地址}$ 。左移 4 位就相当于乘以 16，此时 `eax` 里存放的是当前所在段的物理基地址。

- `add eax, LABEL_SEG_CODE32`

加上 32 位代码段的偏移量（`LABEL_SEG_CODE32`）。此时，`eax` 中存放的就是目标 32 位代码段真正的、完整的 32 位绝对物理地址。

接下来的四行，是将这个 32 位物理地址“塞”进 8 字节长的段描述符结构中。（考点：段描述符的内存布局是被打断的，基地址分为三段存储）。

- `mov word [LABEL_DESC_CODE32 + 2], ax`

段描述符的 Byte 2 和 Byte 3 存放的是基地址的低 16 位（Bit 0-15）。此指令将物理地址的低 16 位（即 `ax`）写入该位置。

- `shr eax, 16`

将 `eax` 右移 16 位。这样处理后，原来的高 16 位被移到了低 16 位的位置。此时 `al` 寄存器存放的是原物理地址的 Bit 16-23，`ah` 寄存器存放的是 Bit 24-31。

- `mov byte [LABEL_DESC_CODE32 + 4], al`

段描述符的 Byte 4 存放的是基地址的中间 8 位。此指令将 `al`（物理地址的 Bit 16-23）写入。

- `mov byte [LABEL_DESC_CODE32 + 7], ah`

段描述符的 Byte 7 存放的是基地址的高 8 位。此指令将 `ah`（物理地址的 Bit 24-31）写入。

总结：

这段代码通过实模式地址转换机制计算出物理首地址，并精准地将其按照 Intel 规定的 GDT 描述符格式，分别填入到了描述符的 Byte 2~3、Byte 4 和 Byte 7 的位置，完

成了 GDT 表项的动态初始化。

1.考察差分时间链：任务T1、T2、T3、T4、T5的延时时间分别为：10、5、7、15、12

(1) 画出当前的差分时间链

(2) 在8个tick之后，任务T6延时了5个tick，画出现在的差分时间链

(3) 用差分时间链的优点？

这是一道考察操作系统底层（特别是 RTOS 定时器管理）差分时间链（Delta List / 差分队列）算法的经典题。

差分时间链的核心逻辑是：将所有按时间排序的任务排成一个链表，每个节点不保存绝对的到期时间，而是保存与前一个节点到期时间的“差值（Delta）”。

以下是详细的解题过程：

(1) 画出当前的差分时间链

第一步：按绝对延时时间对任务进行从小到大排序

- T2: 5
- T3: 7
- T1: 10
- T5: 12
- T4: 15

第二步：计算相邻任务之间的差值（Delta）

- T2: 第一个任务，差值就是它自己的延时 = 5
- T3: $7 - 5 = 2$
- T1: $10 - 7 = 3$
- T5: $12 - 10 = 2$
- T4: $15 - 12 = 3$

第三步：画出当前的差分时间链

链表结构如下（括号内为记录的差值 Delta）：

Head → [T2 : 5] → [T3 : 2] → [T1 : 3] → [T5 : 2] → [T4 : 3] → NULL

(2) 在 8 个 tick 之后，任务 T6 延时了 5 个 tick，画出现在的差分时间链

这个问题需要分两步走：先推演 8 个 tick 后的状态，再将 T6 插入。

第一步：推演 8 个 tick 之后链表的状态

在差分链表中，系统每次 tick 中断只减去头节点的数值。

- **经过 5 个 tick：**头节点 T2 的值减为 0，T2 到期被唤醒并移出链表。此时头节点变成 T3 (值为 2)。
- **再经过 2 个 tick (总计 7 个 tick)：**头节点 T3 的值减为 0，T3 到期被移除。此时头节点变成 T1 (值为 3)。
- **再经过 1 个 tick (总计 8 个 tick)：**头节点 T1 的值减去了 1，变成 $3 - 1 = 2$ 。
- **第 8 个 tick 时的暂存链表：**

Head $\rightarrow$ [T1 : 2] $\rightarrow$ [T5 : 2] $\rightarrow$ [T4 : 3] $\rightarrow$ NULL

(此时 T1 还需要 2 个 tick 到期，T5 还需要 $2+2=4$ 个 tick 到期，T4 还需要 $2 + 2 + 3 = 7$ 个 tick 到期)

第二步：插入任务 T6 (延时 5 个 tick)

T6 的预期延时是 5。我们从头节点开始遍历并相减，寻找插入位置：

1. 看 T1: $5 \geq 2$ ，减去 T1 的差值: $5 - 2 = 3$ 。继续往后找。
2. 看 T5: $3 \geq 2$ ，减去 T5 的差值: $3 - 2 = 1$ 。继续往后找。
3. 看 T4: $1 < 3$ ，不够减了。说明 T6 应该插在 **T5 和 T4 之间**。
4. **计算新差值：**
 - T6 的差值设为刚才剩下的余数，即 1。
 - T4 原本的差值是 3，因为前面插入了 T6 (用掉了 1)，所以 T4 的新差值要减去 1，变成 $3 - 1 = 2$ 。

最终的差分时间链：

Head $\rightarrow$ [T1 : 2] $\rightarrow$ [T5 : 2] $\rightarrow$ [T6 : 1] $\rightarrow$ [T4 : 2] $\rightarrow$ NULL

(3) 差分时间链的优点？

使用差分时间链（相比于普通的绝对时间链表或数组）具有极大的性能优势，尤其是在嵌入式 RTOS 中：

1. **极大地提高了系统时钟中断 (Tick Interrupt) 的处理效率 ($O(1)$ 时间复杂度)：**

在每次系统 Tick 中断到来时，操作系统**不需要遍历整个任务队列，也不需要把所有任务的延时时间都减 1**。内核只需要将链表头节点的差值减 1 即可。因为中断处理函数是最要求短小精悍的，这种设计极大地节省了 CPU 资源，保证了系统的强实时性。

2. 避免了无意义的计算：

只有头节点减到 0 时，才会去处理后续节点，其他节点的差值相对保持静止，这符合“只有最紧迫的任务才需要关注”的逻辑。

3. 节省内存，防止溢出：

不需要保存庞大的绝对时间戳（如 64 位的系统运行总毫秒数），只需要保存较小的相对差值（通常 16 位或 32 位整型即可），降低了时间戳变量溢出导致系统逻辑混乱的风险。

2. 上下文切换

(1) 上下文切换的含义是什么

(2) 哪些时候会进行上下文切换

(3) 结合流程图或者伪代码，对上下文切换的步骤进行说明

这是一道非常核心的操作系统基础题。理解了上下文切换，也就理解了操作系统是如何让单核 CPU “同时”运行多个任务的“戏法”。

以下是对这三个问题的详细解答：

(1) 上下文切换的含义是什么？

上下文 (Context)：指的是一个任务（进程或线程）在 CPU 上运行时的**环境和状态**。具体来说，它主要包括 CPU 各个寄存器的值（如程序计数器 PC、堆栈指针 SP、通用寄存器等）、状态字寄存器（CPSR/PSW），以及内存映射信息（如页表指针）等。

上下文切换 (Context Switch)：是指操作系统内核中止当前正在 CPU 上运行的进程（或线程），将它的这些“状态和环境”保存到内存中（通常是该任务的 PCB/TCB 控制块中），然后从内存中读取另一个被调度到的进程的“状态和环境”，并将其恢复到 CPU 的寄存器中，使得 CPU 能够从该进程上次被中断的地方继续往下执行的过程。

简单来说：**就是 CPU 从“记住并在做任务 A 的事”切换到“记住并在做任务 B 的事”的过程。**

(2) 哪些时候会进行上下文切换？

上下文切换通常由以下三种情况触发：

1. 硬件中断 (Interrupts)：

- **时钟中断（最常见）：** 操作系统为了实现多任务轮转，会设置一个硬件定时器。当分配给当前任务的时间片（Time Slice）用完时，定时器触发中断，强制剥夺当前任务的 CPU 使用权，切换给下一个任务。
- **I/O 中断：** 比如网卡接收到了数据，或者硬盘读取完成，向 CPU 发送中断信号。OS 可能会唤醒正在等待该数据的优先级更高的任务，并触发切换。

2. 系统调用 / 主动让出（System Calls / Voluntarily Yielding）：

- 当前任务需要执行耗时的 I/O 操作（如读取文件 `read()`），或者等待某个信号量/锁（如 `mutex_lock()`）。
- 任务知道自己暂时无法继续运行，会通过系统调用主动陷入内核，将自己挂起（变为阻塞态），系统随之调度其他就绪任务运行。
- 任务主动调用 `sleep()` 或 `sched_yield()` 让出 CPU。

3. 异常（Exceptions）：

- **缺页异常（Page Fault）：** 任务访问的内存数据当前不在物理内存中，需要从硬盘加载。这个过程很慢，OS 会暂停该任务，让它等待内存加载完成，同时切换去运行别的任务。

（3）上下文切换的步骤（伪代码与流程图说明）

上下文切换的核心本质上是寄存器内容的换入和换出。

逻辑流程图

[任务 A 在用户态运行]

|

v (发生中断或系统调用)

[陷入内核态 (硬件自动保存极少量的 PC/CS 等寄存器)]

|

v

[内核：保存任务 A 的完整上下文]

1. 将通用寄存器、状态寄存器等压入 A 的内核栈。
2. 将 A 的栈指针(SP) 保存到 A 的任务控制块(PCB_A)中。

|

v

[内核：执行调度算法 (Scheduler)]

3. 将 A 的状态改为“就绪”或“阻塞”。

4. 从就绪队列中选出下一个要运行的任务 B。
|
v
[内核：恢复任务 B 的上下文]
5. 从 B 的任务控制块(PCB_B)中取出 B 的栈指针(SP)，赋值给 CPU 的 SP 寄存器。
6. 切换内存页表基址寄存器（如果是不同进程间的切换）。
7. 从 B 的内核栈中弹出之前保存的寄存器内容。
|
v (执行中断返回指令 IRET/RETI 等)
[任务 B 恢复在用户态运行]

核心伪代码说明

以下伪代码展示了操作系统的底层上下文切换逻辑（通常由纯汇编编写，类似 Linux 的 `switch_to` 宏）：

```
// 假设 prev 代表当前被剥夺 CPU 的任务，next 代表即将上 CPU 的任务
void context_switch(struct PCB *prev, struct PCB *next) {

    // ----- 阶段 1: 保存 prev 的状态 -----
    ----
    // 1. 保存当前的硬件寄存器（R0-R15，状态寄存器等）到 prev 的
    栈中
    asm("push_all_registers");

    // 2. 将当前的栈顶指针（SP）记录到 prev 的任务控制块中
    prev->kernel_sp = get_current_SP();

    // ----- 阶段 2: 切换内存地址空间 -----
    --
    // 3. 如果 prev 和 next 属于不同的进程（拥有独立内存），则需要
    切换页表
    if (prev->page_table != next->page_table) {
        switch_page_table(next->page_table); // 刷新 TLB 和
        MMU 寄存器
    }

    // ----- 阶段 3: 恢复 next 的状态 -----
```

```

----
// 4. 将 next 任务控制块中保存的栈指针，写回 CPU 的 SP 寄存器
// 此时，CPU 的栈已经变成了 next 任务的栈！
set_current_SP(next->kernel_sp);

// 5. 从 next 的栈中，把之前它挂起时保存的寄存器内容弹出来
asm("pop_all_registers");

// 6. 函数返回。
// 注意：这里的 return 极度巧妙。因为它弹栈返回的地址（PC），
// 已经是 next 任务当初被挂起时保存的地址了！这使得代码直接“穿
越”到了 next 任务。
return;
}

```

关键点总结：上下文切换的“灵魂”在于**堆栈指针（SP）的替换**。一旦把 CPU 的 SP 寄存器指向了新任务的堆栈，CPU 取出的所有历史寄存器数据和返回地址（PC）就全都是新任务的了，从而实现了任务的“无缝切换”。

四、计算题(1题，24分)

1.考察优先级位图法

- (1) 利用C语言，给出OSRdyGrp和OSRdyTbl的定义
- (2) 当有新任务就绪时，应该对OSRdyGrp和OSRdyTbl做哪些操作，用到了TCB的哪些成员？
- (3) 当前优先级位图的状态是OSRdyGrp=0b00000001，OSRdyTbl[0]=0b00010000，OSRdyTbl[1-7]=0b00000000，让你分别描述优先级为7、20、18的任务依次就绪后的优先级位图状态
- (4) 这时候进行调度，最高优先级是多少

这也是嵌入式操作系统（特别是类似 uC/OS-II）中极其经典且必考的核心机制：**基于位图的 \$O(1)\$ 优先级调度算法**。

这道题考察的是你对 64 级优先级就绪表底层数据结构的掌握程度。下面是详细的拆解和解答：

(1) 利用 C 语言，给出 OSRdyGrp 和 OSRdyTbl 的定义

在支持 64 个优先级（0~63）的系统中，只需要 8 位（1 字节）的变量即可映射。定义如下：

C

```
typedef unsigned char INT8U; // 通常使用 8 位无符号整型
```

```
INT8U OSRdyGrp; // 就绪组标志：每一位代表 OSRdyTbl 中的一行（一个组）是否有任务就绪  
INT8U OSRdyTbl[8]; // 就绪表：包含 8 个元素，每个元素的 8 位分别代表该组内的 8 个优先级
```

（2）当有新任务就绪时，对就绪表的操作及用到的 TCB 成员

当一个任务进入就绪态时，需要将其优先级对应位在 `OSRdyTbl` 和 `OSRdyGrp` 中的相应位置 1。

对于优先级 `prio`（范围 0-63），高 3 位（Y）决定组号，低 3 位（X）决定组内偏移。

逻辑操作：

1. 找到任务所在的组（根据高 3 位），把 `OSRdyGrp` 的相应位置 1。
2. 找到任务在组内的具体位置（根据低 3 位），把 `OSRdyTbl[组号]` 的相应位置 1。

用到的 TCB（任务控制块）成员：

为了追求极致的速度，uC/OS-II 会在任务创建时，把这些运算结果提前计算好并存放在 TCB 中，用到以下 4 个成员：

- `OSTCBY`：优先级 `prio` 的高 3 位值，即 `prio >> 3`（决定 `OSRdyTbl` 的数组下标）。
- `OSTCBX`：优先级 `prio` 的低 3 位值，即 `prio & 0x07`。
- `OSTCBBitY`：查表（`OSMapTbl`）得到的组掩码，即把 1 左移 `OSTCBY` 位。
- `OSTCBBitX`：查表得到的位掩码，即把 1 左移 `OSTCBX` 位。

最终的 C 代码操作：

C

```
OSRdyGrp |= ptcb->OSTCBBitY;  
OSRdyTbl[ptcb->OSTCBY] |= ptcb->OSTCBBitX;
```

（3）优先级位图状态的推演

已知初始状态：

- `OSRdyGrp` = `0b00000001`（第 0 组有任务就绪）
- `OSRdyTbl[0]` = `0b00010000`（第 0 组的 Bit 4 为 1，说明当前**优先级 4**已就绪）
- `OSRdyTbl[1-7]` = `0b00000000`

第一步：任务 7 就绪

- 优先级 7 计算： $Y = 7 \div 8 = 0$ ， $X = 7 \pmod{8} = 7$ 。

- 更新 `OSRdyGrp` : Bit 0 已经是 1, 保持不变, 依然是 `0b00000001`。
- 更新 `OSRdyTbl[0]` : 将其 Bit 7 置 1。即 `0b00010000 | 0b10000000 = 0b10010000`。
- 当前状态: `OSRdyGrp=0b00000001`, `OSRdyTbl[0]=0b10010000`

第二步: 任务 20 就绪

- 优先级 20 计算: $Y = 20 \div 8 = 2$, $X = 20 \pmod{8} = 4$ 。
- 更新 `OSRdyGrp` : 将其 Bit 2 置 1。即 `0b00000001 | 0b00000100 = 0b00000101`。
- 更新 `OSRdyTbl[2]` : 将其 Bit 4 置 1。即 `0b00000000 | 0b00010000 = 0b00010000`。
- 当前状态: `OSRdyGrp=0b00000101`, `OSRdyTbl[0]=0b10010000`, `OSRdyTbl[2]=0b00010000`

第三步: 任务 18 就绪

- 优先级 18 计算: $Y = 18 \div 8 = 2$, $X = 18 \pmod{8} = 2$ 。
- 更新 `OSRdyGrp` : Bit 2 已经是 1, 保持不变, 依然是 `0b00000101`。
- 更新 `OSRdyTbl[2]` : 将其 Bit 2 置 1。即 `0b00010000 | 0b00000100 = 0b00010100`。
- 最终状态: `OSRdyGrp=0b00000101`, `OSRdyTbl[0]=0b10010000`, `OSRdyTbl[2]=0b00010100`
(其余全 0)

(4) 此时进行调度, 最高优先级是多少?

寻找最高优先级, 就是寻找就绪表中值最小的有效索引 (即从右往左数, 找到第一个为 1 的位)。

1. 查找最高优先级的组 (Y):

查看最终的 `OSRdyGrp = 0b00000101`。

从右往左 (最低位开始算 Bit 0), 第一个为 1 的位是 **Bit 0**。因此, $Y = 0$ 。

2. 查找该组内最高优先级的位置 (X):

既然确定了最高优先级在第 0 组, 我们就去查看 `OSRdyTbl[0] = 0b10010000`。

从右往左看, 第一个为 1 的位是 **Bit 4**。因此, $X = 4$ 。

3. 计算最高优先级:

$$\text{Highest Priority} = Y \times 8 + X = 0 \times 8 + 4 = 4$$

结论: 这时候进行调度, 最高优先级是 4。

(补充说明: 回顾初始状态 `OSRdyTbl[0]=0b00010000`, 说明早在新任务加入前, 优先级为 4 的任务就已经在那儿等候了, 由于 4 是所有数字中最小的, 所以它依然是最高优先

级任务。)